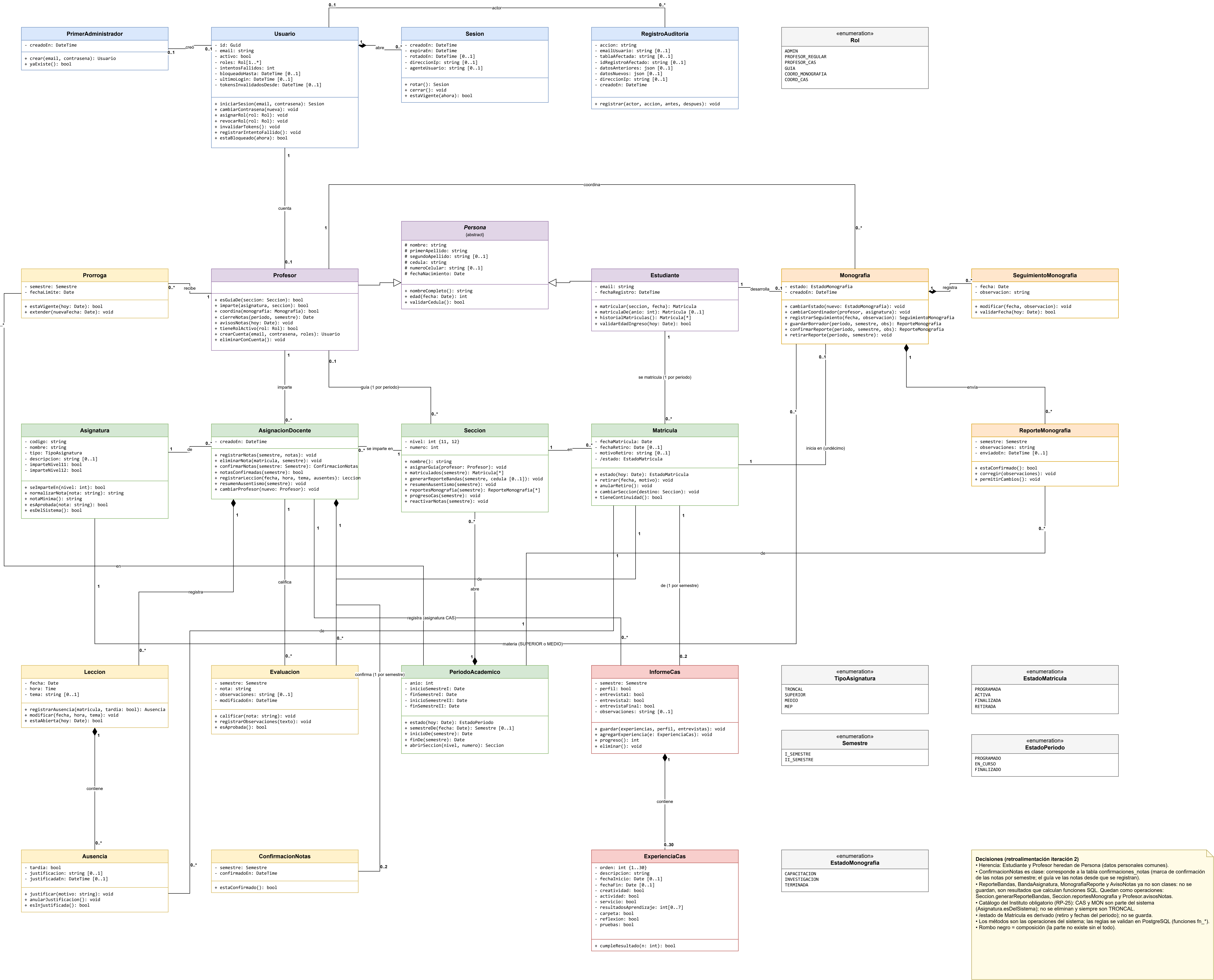
Modelo de dominio — IET BI Portal (iteración 3)

Un solo modelo para todo el sistema. Cada clase persistente corresponde a una tabla del modelo entidad-relación (modelo_entidad_relacion_it3.drawio).


PrimerAdministrador
- creadoEn: DateTime
+ crear(email, contrasena): Usuario
+ yaExiste(): bool
creó
0..1
0..1
Usuario
- id: Guid
- email: string
- activo: bool
- roles: Rol[1..*]
- intentosFallidos: int
- bloqueadoHasta: DateTime [0..1]
- ultimoLogin: DateTime [0..1]
- tokensInvalidadosDesde: DateTime [0..1]
+ iniciarSesion(email, contrasena): Sesion
+ cambiarContrasena(nueva): void
+ asignarRol(rol: Rol): void
+ revocarRol(rol: Rol): void
+ invalidarTokens(): void
+ registrarIntentoFallido(): void
+ estaBloqueado(ahora): bool
0..1
actor
0..*
1
abre
0..*
Sesion
- creadoEn: DateTime
- expiraEn: DateTime
- rotadoEn: DateTime [0..1]
- direccionIp: string [0..1]
- agenteUsuario: string [0..1]
+ rotar(): Sesion
+ cerrar(): void
+ estaVigente(ahora): bool
RegistroAuditoria
- accion: string
- emailUsuario: string [0..1]
- tablaAfectada: string [0..1]
- idRegistroAfectado: string [0..1]
- datosAnteriores: json [0..1]
- datosNuevos: json [0..1]
- direccionIp: string [0..1]
- creadoEn: DateTime
+ registrar(actor, accion, antes, despues): void
«enumeration»
Rol
ADMIN
PROFESOR_REGULAR
PROFESOR_CAS
GUIA
COORD_MONOGRAFIA
COORD_CAS
1
cuenta
0..1
coordina
1
0..*
Persona
{abstract}
# nombre: string
# primerApellido: string
# segundoApellido: string [0..1]
# cedula: string
# numeroCelular: string [0..1]
# fechaNacimiento: Date
+ nombreCompleto(): string
+ edad(fecha: Date): int
+ validarCedula(): bool
Prorroga
- semestre: Semestre
- fechaLimite: Date
+ estaVigente(hoy: Date): bool
+ extender(nuevaFecha: Date): void
0..*
recibe
1
0..*
Profesor
+ esGuiaDe(seccion: Seccion): bool
+ imparte(asignatura, seccion): bool
+ coordina(monografia: Monografia): bool
+ cierreNotas(periodo, semestre): Date
+ avisosNotas(hoy: Date): void
+ tieneRolActivo(rol: Rol): bool
+ crearCuenta(email, contrasena, roles): Usuario
+ eliminarConCuenta(): void
Estudiante
- email: string
- fechaRegistro: DateTime
+ matricular(seccion, fecha): Matricula
+ matriculaDe(anio: int): Matricula [0..1]
+ historialMatriculas(): Matricula[*]
+ validarEdadIngreso(hoy: Date): bool
1
desarrolla
0..1
Monografia
- estado: EstadoMonografia
- creadoEn: DateTime
+ cambiarEstado(nuevo: EstadoMonografia): void
+ cambiarCoordinador(profesor, asignatura): void
+ registrarSeguimiento(fecha, observacion): SeguimientoMonografia
+ guardarBorrador(periodo, semestre, obs): ReporteMonografia
+ confirmarReporte(periodo, semestre, obs): ReporteMonografia
+ retirarReporte(periodo, semestre): void
1
registra
0..*
SeguimientoMonografia
- fecha: Date
- observacion: string
+ modificar(fecha, observacion): void
+ validarFecha(hoy: Date): bool
1
imparte
0..*
0..1
guía (1 por periodo)
0..*
1
se matricula (1 por periodo)
0..*
0..*
0..1
1
envía
0..*
Asignatura
- codigo: string
- nombre: string
- tipo: TipoAsignatura
- descripcion: string [0..1]
- imparteNivel11: bool
- imparteNivel12: bool
+ seImparteEn(nivel: int): bool
+ normalizarNota(nota: string): string
+ notaMinima(): string
+ esAprobada(nota: string): bool
+ esDelSistema(): bool
1
de
0..*
AsignacionDocente
- creadoEn: DateTime
+ registrarNotas(semestre, notas): void
+ eliminarNota(matricula, semestre): void
+ confirmarNotas(semestre: Semestre): ConfirmacionNotas
+ notasConfirmadas(semestre): bool
+ registrarLeccion(fecha, hora, tema, ausentes): Leccion
+ resumenAusentismo(semestre): void
+ cambiarProfesor(nuevo: Profesor): void
0..*
se imparte en
1
Seccion
- nivel: int {11, 12}
- numero: int
+ nombre(): string
+ asignarGuia(profesor: Profesor): void
+ matriculados(semestre): Matricula[*]
+ generarReporteBandas(semestre, cedula [0..1]): void
+ resumenAusentismo(semestre): void
+ reportesMonografia(semestre): ReporteMonografia[*]
+ progresoCas(semestre): void
+ reactivarNotas(semestre): void
1
en
0..*
Matricula
- fechaMatricula: Date
- fechaRetiro: Date [0..1]
- motivoRetiro: string [0..1]
- /estado: EstadoMatricula
+ estado(hoy: Date): EstadoMatricula
+ retirar(fecha, motivo): void
+ anularRetiro(): void
+ cambiarSeccion(destino: Seccion): void
+ tieneContinuidad(): bool
1
inicia en (undécimo)
ReporteMonografia
- semestre: Semestre
- observaciones: string
- enviadoEn: DateTime [0..1]
+ estaConfirmado(): bool
+ corregir(observaciones): void
+ permitirCambios(): void
0..*
de
1
1
1
1
0..*
1
1
1
en
registra
califica
de
abre
de (1 por semestre)
1
registra (asignatura CAS)
de
1
materia (SUPERIOR o MEDIO)
0..*
0..*
0..*
0..*
1
0..*
0..2
Leccion
- fecha: Date
- hora: Time
- tema: string [0..1]
+ registrarAusencia(matricula, tardia: bool): Ausencia
+ modificar(fecha, hora, tema): void
+ estaAbierta(hoy: Date): bool
Evaluacion
- semestre: Semestre
- nota: string
- observaciones: string [0..1]
- modificadoEn: DateTime
+ calificar(nota: string): void
+ registrarObservaciones(texto): void
+ esAprobada(): bool
confirma (1 por semestre)
PeriodoAcademico
- anio: int
- inicioSemestreI: Date
- finSemestreI: Date
- inicioSemestreII: Date
- finSemestreII: Date
+ estado(hoy: Date): EstadoPeriodo
+ semestreDe(fecha: Date): Semestre [0..1]
+ inicioDe(semestre): Date
+ finDe(semestre): Date
+ abrirSeccion(nivel, numero): Seccion
InformeCas
- semestre: Semestre
- perfil: bool
- entrevista1: bool
- entrevista2: bool
- entrevistaFinal: bool
- observaciones: string [0..1]
+ guardar(experiencias, perfil, entrevistas): void
+ agregarExperiencia(e: ExperienciaCas): void
+ progreso(): int
+ eliminar(): void
«enumeration»
TipoAsignatura
TRONCAL
SUPERIOR
MEDIO
MEP
«enumeration»
EstadoMatricula
PROGRAMADA
ACTIVA
FINALIZADA
RETIRADA
«enumeration»
Semestre
I_SEMESTRE
II_SEMESTRE
«enumeration»
EstadoPeriodo
PROGRAMADO
EN_CURSO
FINALIZADO
1
contiene
0..*
1
contiene
0..30
0..*
0..2
Ausencia
- tardia: bool
- justificacion: string [0..1]
- justificadaEn: DateTime [0..1]
+ justificar(motivo: string): void
+ anularJustificacion(): void
+ esInjustificada(): bool
ConfirmacionNotas
- semestre: Semestre
- confirmadoEn: DateTime
+ estaConfirmado(): bool
ExperienciaCas
- orden: int {1..30}
- descripcion: string
- fechaInicio: Date [0..1]
- fechaFin: Date [0..1]
- creatividad: bool
- actividad: bool
- servicio: bool
- resultadosAprendizaje: int[0..7]
- carpeta: bool
- reflexion: bool
- pruebas: bool
+ cumpleResultado(n: int): bool
«enumeration»
EstadoMonografia
CAPACITACION
INVESTIGACION
TERMINADA

Decisiones (retroalimentación iteración 2)
- Herencia: Estudiante y Profesor heredan de Persona (datos personales comunes).
- ConfirmacionNotas es clase: corresponde a la tabla confirmaciones_notas (marca de confirmación de las notas por semestre; el guía ve las notas desde que se registran).
- ReporteBandas, BandaAsignatura, MonografiaReporte y AvisoNotas ya no son clases: no se guardan, son resultados que calculan funciones SQL. Quedan como operaciones: Seccion.generarReporteBandas, Seccion.reportesMonografia y Profesor.avisosNotas.
- Catálogo del Instituto obligatorio (RP-25): CAS y MON son parte del sistema (Asignatura.esDelSistema); no se eliminan y siempre son TRONCAL.
- /estado de Matricula es derivado (retiro y fechas del periodo); no se guarda.
- Los métodos son las operaciones del sistema; las reglas se validan en PostgreSQL (funciones fn_*).
- Rombo negro = composición (la parte no existe sin el todo).